

Comparação Entre Algoritmos de Escalonamento

Breno C. R. Nakamura¹, Estevan T. Santos¹, Gabriel L. Bonavina¹,
Jonas L. Durão¹, Natália V. A. do Val¹, Vinicius A. L. Andrade¹

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
Av. Cesare Monsueto Giulio Lattes, 1201 – 12247-014 – São José dos Campos, SP

Abstract. *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese (“resumo”). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

Resumo. *Este meta-artigo descreve o estilo a ser usado na confecção de artigos e resumos de artigos para publicação nos anais das conferências organizadas pela SBC. É solicitada a escrita de resumo e abstract apenas para os artigos escritos em português. Artigos em inglês deverão apresentar apenas abstract. Nos dois casos, o autor deve tomar cuidado para que o resumo (e o abstract) não ultrapassem 10 linhas cada, sendo que ambos devem estar na primeira página do artigo.*

1. Introdução

O MINIX 3 é um sistema operacional de código aberto criado por Andrew S. Tanenbaum. Ele foi projetado para ser confiável e flexível, sendo utilizado para fins didáticos [Tanenbaum and Woodhull 2008]. Sua arquitetura é baseada em microkernel, na qual apenas as funções mais básicas, como o gerenciamento de interrupções e comunicação entre processos, são executadas no núcleo do sistema. Os demais componentes, como drivers, gerenciamento de arquivos e o escalonador, operam separados do núcleo, como processos independentes no espaço do usuário. Essa separação ajuda a evitar que falhas em um serviço comprometam o sistema inteiro, além de facilitar o estudo e modificação de seu código-fonte.

Este projeto utiliza a versão MINIX 3.4.0rc6 com o objetivo de aplicar na prática os conceitos teóricos da disciplina de Sistemas Operacionais. O trabalho explora o funcionamento interno do sistema por meio de sua instalação em ambiente virtualizado, alteração de chamadas no kernel e, principalmente, através da implementação e análise comparativa de desempenho de diferentes algoritmos de escalonamento de processos sob variadas cargas de trabalho.

2. Instalação

A primeira etapa deste projeto consistiu na instalação e configuração inicial do MINIX na máquina virtual. Para isso, foi necessário seguir o manual de instruções apresentado no enunciado. Imagens da instalação podem ser encontradas nas Figuras 1 e 2.

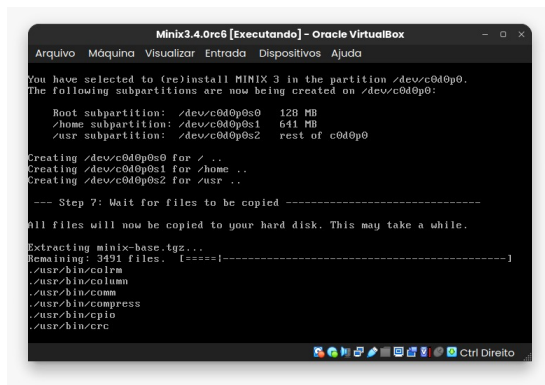


Figura 1. Etapa 7 de instalação do MINIX.

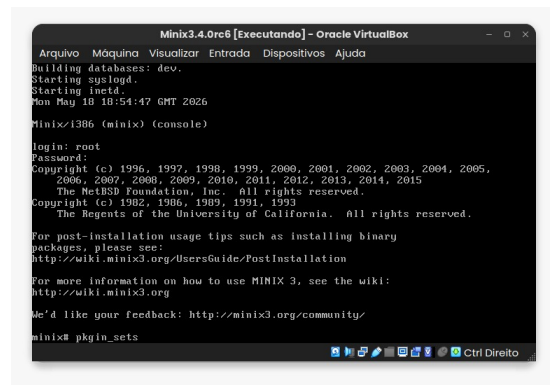


Figura 2. Atualização dos pacotes do MINIX.

Além da instalação do MINIX, também foi necessário fazer a configuração de rede no *Virtual Box*, ajustando a política de endereçamento MAC para incluir apenas os endereços MAC de placas em rede NAT.

Além disso, foi necessário fazer o *clone* do repositório no ambiente virtual, para que fosse possível fazer as modificações que serão detalhadas em seções futuras deste relatório. Para isso, foi necessário configurar um Token de Acesso Pessoal do Github, o que nos permitiu executar os comandos do *git* no ambiente virtual.

Por fim, faz-se necessário ressaltar as configurações de hardware da máquina virtual. Para a execução deste projeto utilizamos uma máquina virtual com 1 *core* de processador, 1GB de memória RAM e 4GB de HD dinamicamente alocado.

3. Mudança nos Banners

Para a mudança no banner durante *boot* e *poweroff*, foi necessário navegar no diretório `minix/minix/kernel/main.c`. Na função `announce()`, foram acrescentados *prints* para reproduzir o banner indicado no enunciado. Além disso, na função `prepare_shutdown()`, também inserimos *prints* para a impressão do banner. Os resultados podem ser encontrados nas Figuras 3 e 4.

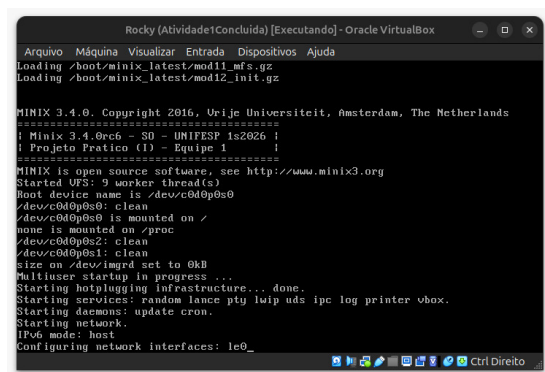


Figura 3. Resultado do banner de boot.

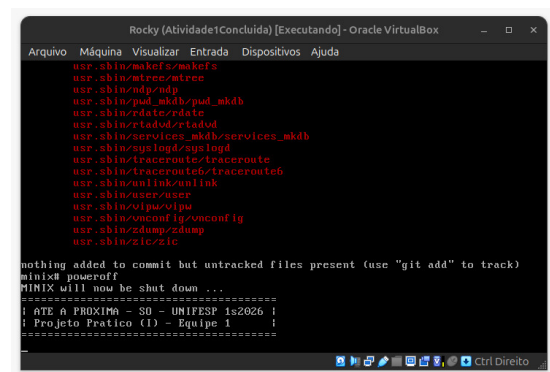


Figura 4. Resultado do banner de poweroff.

Por fim, para a modificação do banner de *login*, foi necessário navegar pelos diretórios `minix/etc/motd`, onde inserimos o banner indicado pelo enunciado. O resultado da modificação pode ser encontrado na Figura 5.

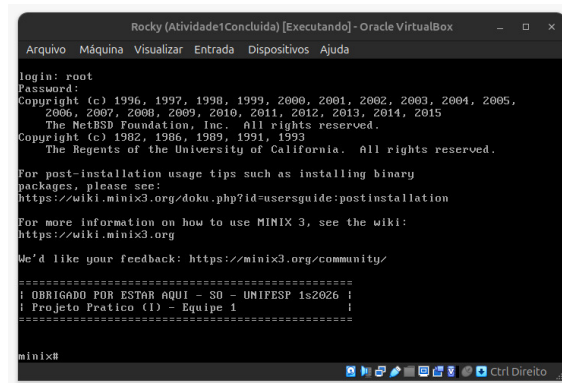


Figura 5. Resultado do banner de *login*.

4. Mudança `exec.c`

Para registrar a execução de processos exibindo o comando e seu caminho absoluto, a implementação foi dividida entre o *Virtual File System* (VFS) e o *Process Manager* (PM), respeitando a modularidade da arquitetura do MINIX [MINIX 3 Project].

Inicialmente, adicionou-se o campo `execpath` à estrutura compartilhada `exec_info` (no arquivo `libexec.h`) como um campo de comunicação entre o VFS e o PM. No VFS (`vfs/exec.c`), logo após o carregamento do executável, o caminho absoluto armazenado na variável `finalexec` é copiado para `execi.args.execpath` utilizando a função `strncpy`. Em seguida, o VFS aciona o PM por meio de `libexec_pm_newexec`.

A exibição da mensagem foi centralizada no PM (`pm/exec.c`). Na rotina `do_newexec`, os dados preenchidos pelo VFS são recebidos via `sys_datacopy`. Após a cópia dos argumentos para a memória do PM, inseriu-se um `printf` para exibir a mensagem Executando: `<caminho/comando>`. O resultado dessa modificação pode ser encontrado na Figura 6.

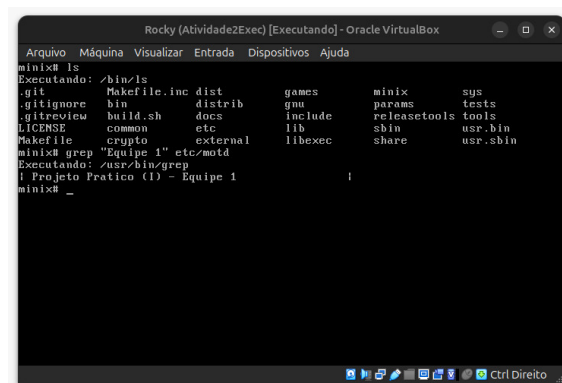


Figura 6. Resultado da modificação do `exec.c`.

5. Descrição do Exercício Proposto

Este projeto tem como objetivo avaliar o desempenho de diferentes algoritmos de escalonamento no sistema operacional MINIX 3.4.0rc6. A metodologia consiste na análise teórica e na modificação do código-fonte do sistema para implementar três políticas de escalonamento distintas, além da abordagem padrão do MINIX. Para isso, foram implementados os algoritmos: Round-Robin, Múltiplas Filas com *Quantum* Exponencial e *First-Come, First-Served* (FCFS).

Para analisar o comportamento e a eficiência de cada algoritmo sob diferentes perfis de carga, foram conduzidos testes comparativos simulando cenários com processos intensivos em computação (*CPU-bound*) e intensivos em entrada/saída (*I/O-bound*). A avaliação quantitativa será realizada por meio do programa de testes `teste_processos.c`, variando-se a carga de trabalho em escalas de 10, 50, 100 e 200 processos simultâneos. Adicionalmente, o sistema será avaliado em diferentes cenários, submetendo os escalonadores à execução de 10 mil processos *I/O-bound* e 1 milhão de processos *CPU-bound*.

6. Descrição dos Algoritmos de Escalonamento

6.1. Algoritmo Padrão do MINIX

O algoritmo de escalonamento padrão do MINIX 3 utiliza um escalonador preemptivo baseado em múltiplas filas de prioridade com ajuste dinâmico. O sistema gerencia 16 filas (0 a 15), nas quais o nível 0 possui a maior prioridade e o nível 15, a menor. O escalonador sempre seleciona o processo localizado no topo da fila de maior prioridade que não esteja vazia. As filas superiores, de 0 a 6, são destinadas a processos críticos do núcleo e a servidores essenciais do sistema. Já os processos de usuário são alocados entre as filas 7 (`USER_Q`) e 14 (`MIN_USER_Q`). A fila 15 (`IDLE_Q`), de menor prioridade, é reservada ao processo ocioso do sistema, executado apenas quando não há nenhuma outra instrução pendente no processador.

Devido à arquitetura de microkernel do MINIX 3, o escalonamento ocorre tanto no espaço de núcleo quanto no espaço de usuário. No espaço de núcleo, implementado principalmente em `/minix/kernel/proc.c`, encontram-se os mecanismos de baixo nível, fornecendo as operações `enqueue()` e `dequeue()`, responsáveis pela manipulação das estruturas de dados, além da função `pick_proc()`, responsável por selecionar o próximo processo a receber a CPU com base na prioridade estrita.

Por outro lado, a política de escalonamento é implementada no espaço de usuário no diretório `/minix/servers/sched/`, com destaque para o arquivo `schedule.c`, onde o servidor de escalonamento `sched` gerencia o tempo de execução dos processos. Internamente, cada fila utiliza o algoritmo Round-Robin, no qual cada processo recebe uma fatia limitada de tempo de CPU (*quantum*). O servidor `sched` reduz a prioridade de processos pesados (*CPU-bound*) que esgotam seu *quantum* e promove processos interativos (*I/O-bound*) por meio do mecanismo de *aging*, implementado na função `balance_queues()`, garantindo maior responsividade e evitando inanição.

6.2. Round-Robin (RR)

O algoritmo de escalonamento por chaveamento circular (ou *Round-Robin*), é um dos algoritmos mais simples e antigos devido à sua fácil implementação. Seu funcionamento

gira em torno da definição de uma fatia de tempo de CPU (*quantum*), em que o processo que está na CPU terá um tempo limitado de uso e, ao final deste tempo, haverá preempção e um novo processo será escolhido para ser executado [Tanenbaum 2016, p. 109–110].

A implementação concentrou-se em `minix/servers/sched/schedule.c`, sem alterar o *kernel*. O *quantum* definido foi de $\frac{1000}{n} ms$, com n dado por `count_ready_user_queue()` (processos de usuário prontos em `USER_Q`, via `sys_getinfo(GET_PROC)` e `proc_is_runnable()`). Foram incluídos `<minix/timers.h>` e `"kernel/proc.h"` e criadas `apply_variable_quantum()` e `reschedule_all_user_quantum()`; em `do_start_scheduling()`, `do_stop_scheduling()` e `do_noquantum()` o tempo de CPU é recalculado e aplicado com `sys_schedule()`. Processos de sistema (`is_system_proc`) mantêm prioridade própria; os restantes ficam fixos em `USER_Q`.

Para a implementação do RR ser feita sem mudar as funções `enqueue()`, `dequeue()` e `pick_proc()` (`minix/kernel/proc.c`), todos os usuários são forçados a adotar a fila `USER_Q` 7 em `do_start_scheduling()` e `do_nice()`; a rotação na cauda ao expirar o *quantum* ou ao desbloquear por E/S equivale ao *Round Robin* entre processos nessa fila. Em `minix/servers/pm/utility.c`, `nice_to_priority()` passa sempre `*new_q = USER_Q`, pelo que `nice()` deixa de distribuir processos pelas 16 filas. **Confuso**

Removeu-se a política MLFQ no SCHED em `do_noquantum()` eliminou-se `rmp->priority += 1`. `balance_queues()` e `init_scheduling()` ficaram vazias e em `main.c` deixou de se chamar `balance_queues()` nas notificações do CLOCK. Ao expirar o *quantum*, o processo permanece em `USER_Q`, recalcula a fatia com `apply_variable_quantum()` e reagenda-se com `schedule_process_local()` (reinserção na cauda no kernel via `sched_proc`).

6.3. Múltiplas Filas com Quantum Exponencial

O segundo escalonamento implementado consiste em uma alteração na política adotada pelo algoritmo de múltiplas filas padrão do MINIX. No algoritmo padrão do MINIX 3.4.0rc6, utiliza-se um *quantum* fixo para cada processo de usuário, independentemente de sua fila. A modificação desenvolvida baseia-se no modelo do sistema de tempo compartilhado CTSS, implementado no IBM 7094, conforme descrito por Tanenbaum ([Tanenbaum 2016, p. 111]). Nela, parte-se do pressuposto de que é mais eficiente conceder fatias de tempo de CPU progressivamente maiores para determinados processos, em vez de assegurar períodos curtos e frequentes, reduzindo o *overhead* causado pelo excesso de trocas de contexto. Sendo assim, o projeto estabelece que sempre que um processo esgotar o *quantum* a ele alocado, ele seja rebaixado para a fila de prioridade imediatamente inferior, recebendo um novo *quantum* cujo valor cresce exponencialmente, em potências de dois, de acordo com a sua nova fila. Para a última fila, os processos remanescentes passam a ser escalonados segundo a política *Round-Robin* padrão, alternando com o *quantum* correspondente àquele nível.

Para implementar as alterações propostas, as modificações foram feitas no arquivo `minix/servers/sched/schedule.c`. Vale ressaltar que as novas regras de escalonamento aplicam-se exclusivamente aos processos de usuário, ou seja, aqueles que estão em filas acima de `USER_Q` 7. Para uma implementação coerente, definiu-se

uma nova fatia de tempo inicial correspondente a 10% do valor padrão que o MINIX 3.4.0rc6 define para processos de usuário. Isso foi feito a partir da modificação do valor de `DEFAULT_USER_TIME_SLICE` para o valor de 20 no lugar de 200. Isso permitiu verificar nos resultados a influência do crescimento progressivo do tempo de CPU para processos longos.

Com o objetivo de desenvolver essa lógica, a função `user_quantum_for_priority` foi criada para calcular, em potências de dois, o *quantum* correspondente a cada nível de fila. O controle do tempo de CPU atribuído a cada processo, por sua vez, é atualizado pela função `update_user_quantum`. A chamada dessa função ocorre em situações distintas: em `do_start_scheduling()`, para inicializar o processo com o *quantum* base; em `do_noquantum()`, para atualizar o *quantum* no momento da preempção por esgotamento do tempo de CPU do processo; no caso `SCHEDULING_INHERIT`, para assegurar que o filho, criado na mesma fila de prioridade que o pai, tenha seu *quantum* corretamente definido; e, por fim, em `do_nice()`, para que a mudança de prioridade ajuste a fatia de tempo adequada.

Para assegurar o comportamento definido no projeto, o rebalanceamento das filas executado periodicamente em determinados ciclos de *clock* foi desativado. Para isso, em `init_scheduling()` os trechos que fazem a chamada de `sys_setalarm(balance_timeout)` foram comentados, de modo que em `minix/servers/sched/main.c` não haja a invocação de `balance_queues`.

6.4. *First-Come, First-Served (FCFS)*

O algoritmo *First-Come, First-Served* atende os processos estritamente na ordem de chegada, sem preempção por tempo. Sua implementação no MINIX 3 exigiu desativar a prioridade dinâmica de múltiplas filas e as interrupções temporais para as tarefas de usuário, garantindo que executem de forma exclusiva até sua finalização ou bloqueio por E/S.

No espaço de núcleo, as modificações foram realizadas no arquivo `/minix/kernel/proc.c`, onde a preempção foi desabilitada na função `proc_no_time()`. A chamada `notify_scheduler(p)`, que efetuava o dequeue do processo ao fim do tempo, foi substituída pela renovação contínua da sua fatia de tempo (`p->p_cpu_time_left`). Essa abordagem mantém a CPU com o processo atual sem quebrar os cálculos estatísticos internos do microkernel com um *quantum* infinito.

No espaço de usuário, as alterações foram realizadas no arquivo `/minix/servers/sched/schedule.c`. A estabilidade do sistema foi assegurada mantendo as filas 0 a 6 inalteradas para os servidores críticos. O FCFS foi aplicado exclusivamente aos processos do usuário. Na função `do_start_scheduling()`, todo processo com prioridade igual ou superior a 7 passou a ser alocado forçadamente em uma única fila (`USER_Q`). Com todos os processos de usuário concentrados nessa fila e sem variação de prioridade, os mecanismos de rebaixamento implementados em `do_noquantum()` e de promoção por *aging* implementados em `balance_queues()` foram completamente removidos. Por fim, para evitar interferências externas, a função `do_nice()` foi modificada para retornar `OK` imediatamente, impedindo a alteração manual de prioridades pelos usuários.

7. Resultados Obtidos e Discussão

Para garantir a robustez estatística de nossos resultados, realizaram-se 5 execuções para cada algoritmo, e calcularam-se a média e o desvio-padrão de cada um deles. Ao final das execuções, foi desenvolvido um gráfico de barras para cada classe de processos, no qual o eixo X representa o número de processos e o eixo Y representa o tempo de execução médio. Esses resultados são apresentados na Figura 7 e na Figura 8. Neles são apresentados em azul o algoritmo padrão do MINIX, em amarelo o FCFS, em verde o *Round-Robin* (RR) e em vermelho o Múltiplas Filas com *Quantum* Exponencial (MF).

Também é válido ressaltar que todos os testes de desempenho foram executados no mesmo computador, o qual foi mantido sem a execução de tarefas paralelas ou uso pessoal, de modo que os resultados obtidos fossem consistentes. A configuração do computador utilizado foi um processador 12th Gen Intel(R) Core(TM) i5-12500H com 16GB de RAM, que estava com o Windows 11 instalado. Por fim, a versão da Oracle VirtualBox é a 7.2.6.

7.1. Resultados para testes de processos *CPU-bound*

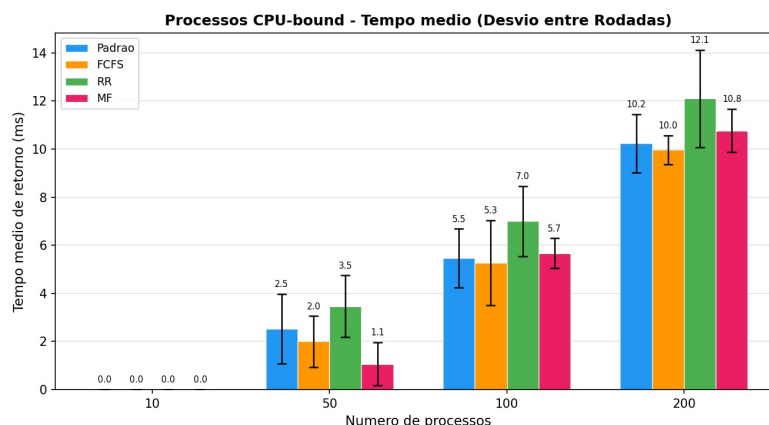


Figura 7. Resultados obtidos para processos *CPU-bound*.

Partindo pela análise dos resultados para os processos *CPU-bound*, apresentado na Figura 7, observa-se, de maneira geral, que o *Round-Robin* teve o pior desempenho dentre todos os métodos de escalonamento a partir dos testes com 50 processos. **TALVEZ COLOCAR UMA EXPLICAÇÃO GERAL DO POR QUÊ COMO UM RESULTADO IMEDIATO.**

Avaliando cada teste individualmente, foi possível verificar que, com uma carga base de 10 processos, o tempo de retorno registrado é de 0.0 ms para todos os algoritmos. É importante destacar que esse valor não indica um tempo de execução nulo, mas sim que a duração foi tão curta que ficou abaixo do limite de resolução rastreável pelo mecanismo de medição do sistema. Ou seja, o sistema executou todos os processos dessa categoria sem nenhuma dificuldade, com todos os algoritmos sendo capazes de concluir as tarefas de forma quase instantânea.

As divergências entre os algoritmos passaram a ser nítidas na medida em que se aumentou a carga de processos. Para 50 processos, o algoritmo MF se destaca com o menor tempo médio de retorno, obtendo o valor de 1.1 ms. O seu funcionamento pode

explicar esse comportamento. Nesse algoritmo, para processos *CPU-bound*, ocorre um escalonamento sem pausas de execução, de modo que os *quantum* dos processos são estourados, resultando na preempção com o rebaixamento de fila.

Sendo assim, para 50 processos, observou-se que a sobrecarga inicial, que ocorre em função das trocas de contexto nas filas superiores devido ao *quantum* pequeno, foram compensadas pela acomodação desses processos em filas mais baixas. Dessa forma, as tarefas de uso intensivo de CPU puderam ser executadas por fatias de tempo maiores sem novas trocas de contexto, finalizando a execução de maneira muito mais rápida.

Por outro lado, na medida que o sistema é levado a cenários de alta concorrência, com o número de processos chegando a um total de 100 e 200, o algoritmo MF teve uma queda de rendimento, ficando à frente apenas do RR. Ao dobrar ou quadruplicar a carga, o funcionamento do MF perdeu eficiência em razão do grande volume de trocas de contexto exigido nas primeiras filas. Esse *overhead* inicial maior não conseguiu ser compensado pela estabilidade posterior nas filas inferiores. Além disso, as últimas filas, que executam processos únicos por longos períodos, são suscetíveis a acumular uma extensa sequência de processos no estado de pronto, gerando longos tempos de espera e o risco de *starvation*.

Nesses cenários, os algoritmos FCFS e o padrão do MINIX demonstram melhor escalabilidade e estabilidade **JUSTIFICATIVA BRENO**. No teste mais extremo de 200 processos, o FCFS obtém a melhor marca com 10.0 ms, seguido rigorosamente pelo escalonador Padrão com 10.2 ms. **O desempenho superior do FCFS nestas condições se justifica teoricamente pela sua natureza não preemptiva: processos limitados por CPU executam até o fim sem interrupções forçadas, eliminando o overhead gerado por constantes trocas de contexto breno verificar informacao**. Em contrapartida, o algoritmo Round Robin (RR) sofre o maior impacto sob carga pesada, atingindo 12.1 ms e exibindo a maior variância (desvio-padrão) entre as rodadas. Isso ocorre devido ao número de processos na fila, pois o *quantum* com fração justa será muito menor ($\frac{1000}{200} = 5\text{ ms}$), resultando em um processo que em execução ficará pouco tempo na CPU, sendo necessário muitas trocas de contexto. **Obs: quando inserir a parte do FCFS, reajustar a estrutura do texto para explicar com coesão ambos os algoritmos**

Um ponto válido a ser levantado acerca da Figura 7 é que, o algoritmo padrão do MINIX para 200 processos, mesmo sendo o segundo melhor algoritmo, é possível notar que em algumas execuções ele foi capaz de atingir performances superiores aos demais. **Discorrer**.

7.2. Resultados para testes de processos *IO-bound*

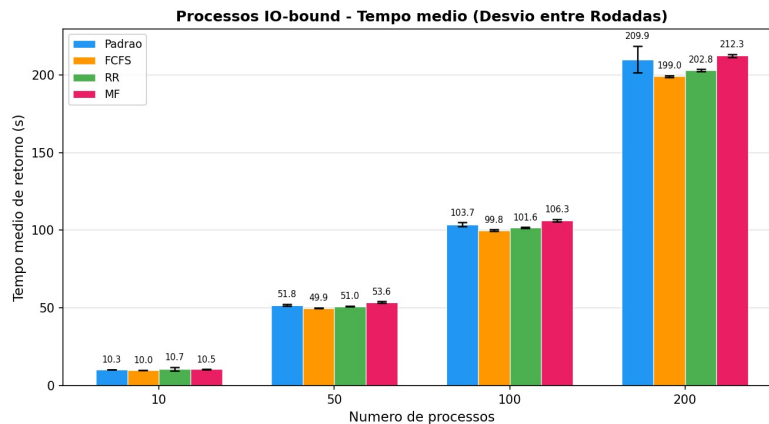


Figura 8. Resultados obtidos para processos *IO-bound*

Agora, a análise será direcionada para os processos *IO-bound*, com resultados apresentados na Figura 8. Note que, nesse caso, os tempos médios de retorno operam na ordem dos segundos. Isso reflete a natureza desse tipo de processo, que consiste em passar a maior parte do seu tempo de vida esperando a conclusão de operações de leitura ou escrita em dispositivos periféricos.

A respeito dos algoritmos de escalonamento, de imediato, observa-se que o comportamento dos algoritmos foi mais **uniforme comparado ao resultado dos processos *CPU-bound*** COMENTÁRIO: PODEMOS AFIRMAR ISSO NUMA ESCALA DE SEGUNDOS? DEVEMOS DISCORRER SOBRE ISSO?.

Avaliando cada um dos algoritmos individualmente, é notável que o FCFS apresentou a melhor performance em todas as faixas de teste em relação aos demais, com seu tempo médio de retorno variando de 10.0s com 10 processos até 199.0s com 200 processos. **Por que?.**

Seguido do FCFS, o *Round-Robin* foi, de maneira geral, a segunda melhor alternativa para I/O intenso, visto que, à medida que o número de processos aumentava, seus resultados mantiveram-se consistentemente em segundo lugar, apresentando um tempo médio de retorno de 202.8s em seu pior cenário, destacando-se em relação aos dois algoritmos restantes. Isso ocorre em função da característica inerente desse tipo de processo, já que o tamanho do *quantum* não é relevante para a sua execução. Como as tarefas possuem uma alta frequência de E/S, o tempo total passa a depender da espera por essas operações. Sempre que a requisição é concluída, a tarefa é enviada ao final da fila de prontos, o que torna o comportamento do escalonador muito similar ao do FCFS, justificando a semelhança no tempo médio de ambos os algoritmos.

Por outro lado, algoritmos baseados em prioridades dinâmicas mostraram-se menos favoráveis para esse perfil específico de carga. O algoritmo MF, por exemplo, registrou o tempo de retorno mais lento no cenário de alta concorrência com 200 processos, atingindo a marca de 212.3 s. Essa degradação de desempenho ocorre porque tarefas limitadas por I/O realizam bloqueios voluntários com muita frequência, alternando constantemente entre os estados de bloqueado e pronto para, em seguida, retornarem ao final da

fila. Consequentemente, o algoritmo de Múltiplas Filas com *Quantum* Exponencial acaba não entregando um bom desempenho porque o seu projeto é feito fundamentalmente para beneficiar processos longos e de uso intensivo de processador, através do ajuste dinâmico de prioridades e adequação do *quantum*. Sendo assim, o mecanismo do MF não oferece nenhuma vantagem considerável para esse tipo de tarefa. Na prática, esses processos acabam circulando ineficientemente nas filas superiores de forma repetitiva ou demoram para ter seu tempo de CPU realocado de maneira adequada ao seu perfil, o que prejudica a fluidez geral da execução.

O mesmo princípio se aplica ao algoritmo padrão do MINIX. Como o rebaixamento nesse escalonador depende do esgotamento de um quantum fixo, os processos limitados por I/O, que costumam bloquear voluntariamente antes desse limite, são rebaixados com menos frequência. Como resultado, em cenários de alta concorrência, a grande maioria desses processos pode se concentrar em uma mesma fila de alta prioridade. Isso gera um congestionamento, elevando consideravelmente os tempos de espera devido à longa fila de tarefas que, ao serem recém-acordadas, retornam ao final da fila.

Referências

- MINIX 3 Project. Minix 3 wiki documentation. <https://wiki.minix3.org/>. Accessed: 2026-06-04.
- Tanenbaum, A. S. (2016). *Sistemas Operacionais Modernos*. São Paulo: Pearson Education Brasil, 4th edition.
- Tanenbaum, A. S. and Woodhull, A. S. (2008). *Sistemas Operacionais: Projeto e Implementação*. Porto Alegre: Bookman, 3rd edition.